TITLE: - SUM OF 1 TO 10

AIM: -

To write a ALP to sum numbers from 1 to 10 and display the result on screen.

ALGORITHM: -

- Initialize data segment register.
- Set CX = 10 and AX = 0.
- Add CX to AX and repeat using LOOP until CX = 0.
- Save the sum on stack.
- Display the message string.
- Restore the sum into AX.
- Convert the sum to decimal by repeated division by 10 and store digits on stack.
- Pop digits, convert to ASCII, and display them.

PROGRAM: -

```
TITLE     SUM.ASM -- Program to sum of 1 to 10
;//**************************************************************************
;//* *
;//* Copyright (c) 1985-2022, American Megatrends International LLC.      *
;//* *
;//* All rights reserved. Subject to AMI licensing agreement.             *
;//* *
;//**************************************************************************
;<AMI_FHDR_START>
;
; Header:
```

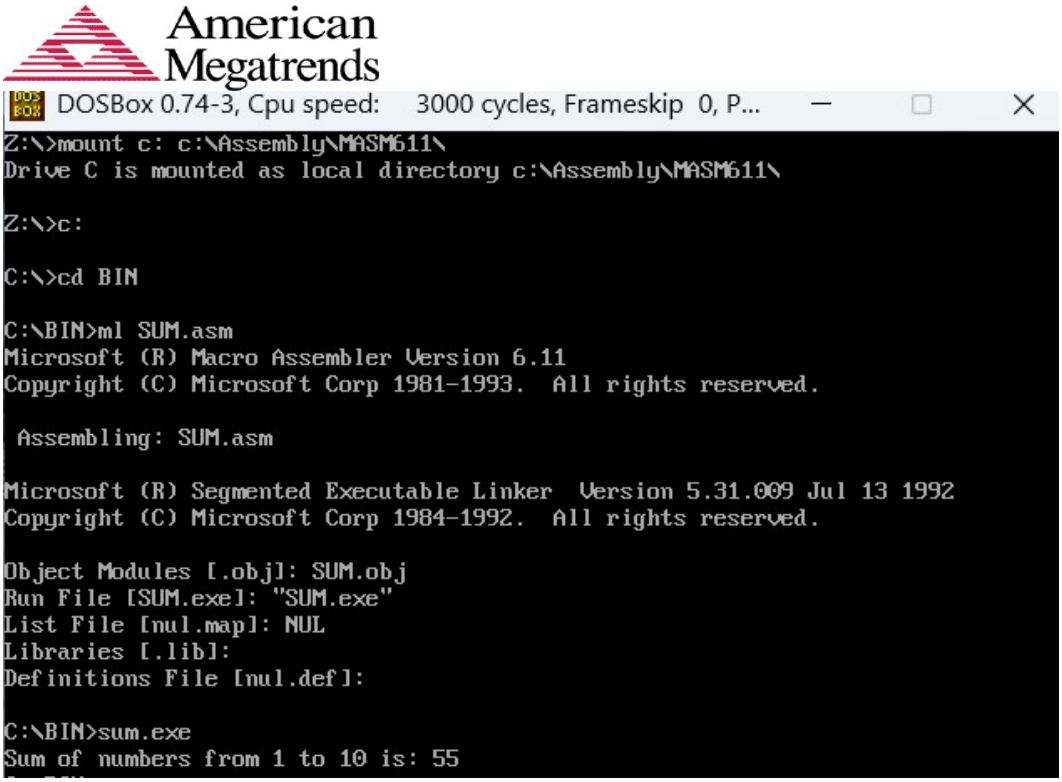
```
; Author: Muthuganesh s
; Description: Program to sum numbers from 1 to 10 and display result
; Date: 28/03/2026
;
;<AMI_FHDR_END>
;*************************************************************************

.model small
.stack 100h

;-------------------------------------------------------------------------
;               DATA SEGMENT
;-------------------------------------------------------------------------
data segment

msg db "Sum of numbers from 1 to 10 is: $"

data ends
;-------------------------------------------------------------------------

;-------------------------------------------------------------------------
;               CODE SEGMENT
;-------------------------------------------------------------------------
code segment
assume cs:code, ds:data

start:
```

```
        ; Initialize Data Segment
        mov ax, data
        mov ds, ax

        ;---------------------------------------
        ; Initialize registers
        ;---------------------------------------
        mov cx, 10              ; Counter (10 numbers)
        mov ax, 0               ; AX will store sum

        ;---------------------------------------
        ; Loop to calculate sum
        ;---------------------------------------
sum_loop:
        add ax, cx              ; AX = AX + CX
        loop sum_loop           ; CX = CX - 1, repeat until CX = 0

        ; At this point AX = 55

        ; ---> FIX ADDED HERE: Save the sum before printing the message
        push ax

        ;---------------------------------------
        ; Display message
        ;---------------------------------------
        mov dx, offset msg
```

```
        mov ah, 09h
        int 21h

        ; ---> FIX ADDED HERE: Restore the sum back into AX
        pop ax

        ;----------------------------------------
        ; Convert number (AX) to decimal digits
        ;----------------------------------------
        mov bx, 10              ; Divisor = 10
        mov cx, 0               ; Digit counter

convert_loop:
        xor dx, dx              ; Clear DX before division
        div bx                  ; AX ÷ 10 → Quotient in AX, Remainder in DX

        push dx                 ; Save remainder (digit)
        inc cx                  ; Count digits

        cmp ax, 0
        jne convert_loop        ; Continue until quotient = 0

        ;----------------------------------------
        ; Display digits (reverse order)
        ;----------------------------------------
print_loop:
        pop dx                  ; Get digit
```

```
    add dl, '0'             ; Convert to ASCII

    mov ah, 02h
    int 21h

    loop print_loop

    ;---------------------------------------
    ; Exit program
    ;---------------------------------------
    mov ah, 4Ch
    int 21h

code ends
end start
;-------------------------------------------------------------------------
```

OUTPUT: -



TITLE: - FIND A CHARACTER IN STRING

AIM: -

To write program to find a character in string using ALP.

ALGORITHM:-

- Initialize the data segment register.
- Display prompt to enter a string.
- Read the string using DOS function 0Ah.
- Display prompt to enter a character.
- Read a single character using DOS function 01h.
- Store the input character in a variable.
- Initialize pointer to the start of the input string.
- Compare each character of the string with the search character.
- If match is found, display “Character Found”.

- If end of string (Carriage Return) is reached, display “Character Not Found”.

PROGRAM:-

```
TITLE     CHAR.ASM -- Program to find a character in a user-input string


;//**************************************************************************
;//* *
;//* Copyright (c) 1985-2022, American Megatrends International LLC.      *
;//* *
;//* All rights reserved. Subject to AMI licensing agreement.             *
;//* *
;//**************************************************************************
;<AMI_FHDR_START>
;
; Header:
; Author: Muthuganesh s
; Description: Program to take string and character input and search
; Date: 28/03/2026
;
;<AMI_FHDR_END>
;**************************************************************************


.model small
.stack 100h
```

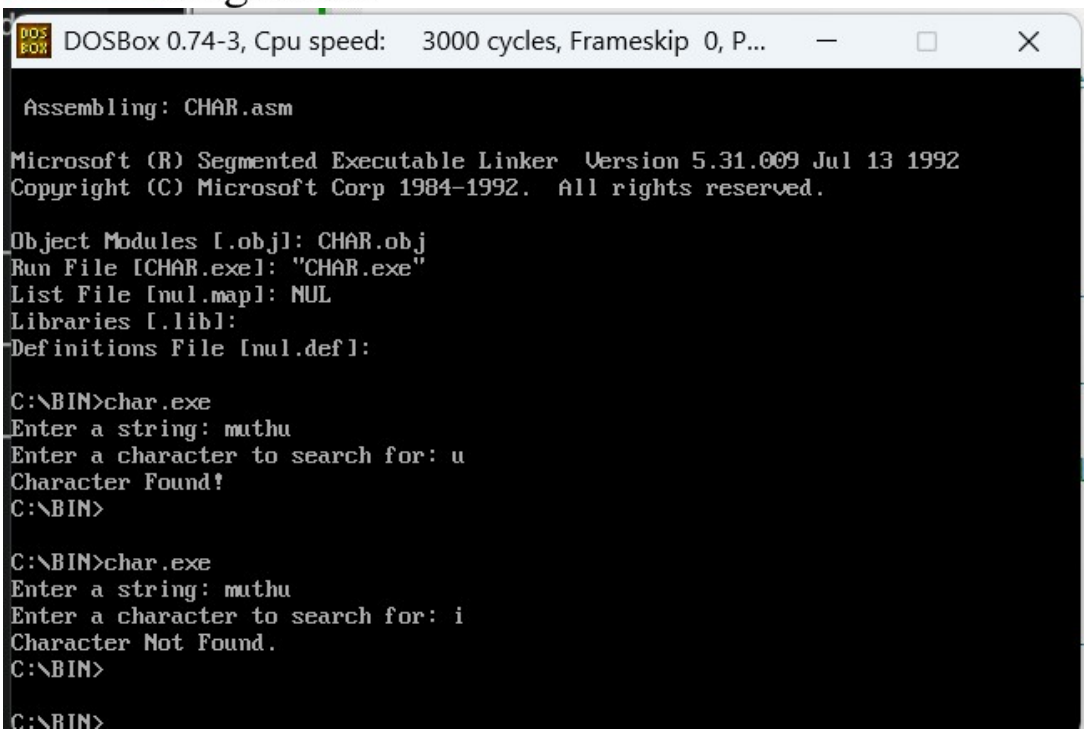
```
;---------------------------------------------------------------------------
;               DATA SEGMENT
;---------------------------------------------------------------------------
data segment

        prompt1 db "Enter a string: $"
        prompt2 db 13, 10, "Enter a character to search for: $"

        msg1 db 13, 10, "Character Found!$"
        msg2 db 13, 10, "Character Not Found.$"

        ;-------------------------------------------------
        ; Buffer structure for reading a string via AH=0Ah
        ;-------------------------------------------------
        input_buffer db 50              ; Byte 0: Max characters allowed to read
        actual_len      db ?            ; Byte 1: Actual characters read (filled by DOS)
        user_string     db 50 dup(?)    ; Byte 2+: The actual string data entered
        search_char     db ?            ; Variable to hold the single character
data ends
;---------------------------------------------------------------------------
;               CODE SEGMENT
;---------------------------------------------------------------------------
code segment
assume cs:code, ds:data
start:
```

```
; Initialize Data Segment
mov ax, data
mov ds, ax
;-------------------------------------------------
; 1. Prompt user for string
;-------------------------------------------------
mov dx, offset prompt1
mov ah, 09h
int 21h


;-------------------------------------------------
; 2. Read string from user
;-------------------------------------------------
mov dx, offset input_buffer
mov ah, 0Ah
int 21h
;-------------------------------------------------
; 3. Prompt user for character
;-------------------------------------------------
mov dx, offset prompt2
mov ah, 09h
int 21h
;-------------------------------------------------
; 4. Read single character from user
;-------------------------------------------------
```

```
    mov ah, 01h
    int 21h
    mov search_char, al          ; Save the typed character into our variable
    ;------------------------------------------------
    ; 5. Initialize for Search
    ;------------------------------------------------
    lea si, user_string          ; Point SI to the start of the typed string
    mov al, search_char          ; Load the character we are looking for into AL
search_loop:
    mov bl, [si]                 ; Load current character from string

    cmp bl, 0Dh                  ; Did we hit Carriage Return? (0Dh is the end of user input)
    je not_found                 ; If yes, we reached the end without finding it
    cmp bl, al                   ; Compare string character with search character
    je found                     ; If equal, jump to found
    inc si                       ; Move to next character
    jmp search_loop              ; Repeat loop
    ;------------------------------------------------
    ; If character is found
    ;------------------------------------------------
found:
    mov dx, offset msg1          ; Load "Found" message
    mov ah, 09h
    int 21h
    jmp exit_program
```

```
        ;-----------------------------------------------

        ; If character is not found

        ;-----------------------------------------------

not_found:

        mov dx, offset msg2             ; Load "Not Found" message

        mov ah, 09h

        int 21h

        ;-----------------------------------------------

        ; Exit program

        ;-----------------------------------------------

exit_program:

        mov ah, 4Ch

        int 21h

code ends

end start

;-------------------------------------------------------------------------
```

OUTPUT:-.



TITLE: - REVERSE A STRING

AIM: -

To write a ALP program to reverse a string.

ALGORITHM:-

- Initialize the data segment register.
- Display prompt to enter a string.
- Read the string using DOS function 0Ah.
- Check if the string length is zero; if yes, terminate.
- Load string length into CX and point to string start.
- Push each character of the string onto the stack.
- Reset pointer and counter to the beginning of the string.
- Pop characters from the stack and store back into the string.
- Append $ at the end of the reversed string.
- Display result message.
- Display the reversed string.

PROGRAM:

```
TITLE     REV.ASM -- Program to reverse a string


;//****************************************************************************
;//* *
;//* Copyright (c) 1985-2022, American Megatrends International LLC.      *
;//* *
;//* All rights reserved. Subject to AMI licensing agreement.             *
;//* *
;//****************************************************************************
;<AMI_FHDR_START>
;
; Header:
```

```
; Author: Muthuganesh s

; Description: Program to take a user string, reverse it using the stack, and display it.

; Date: 02/04/2026

;

;<AMI_FHDR_END>

;*************************************************************************


.model small

.stack 100h


;-------------------------------------------------------------------------

;               DATA SEGMENT

;-------------------------------------------------------------------------

data segment


        prompt db "Enter a string to reverse: $"

        msg        db 13, 10, "Reversed string: $"


        ;-----------------------------------------------

        ; Buffer structure for reading a string via AH=0Ah

        ;-----------------------------------------------

        input_buffer db 50                  ; Byte 0: Max characters allowed

        actual_len      db ?                    ; Byte 1: Actual characters typed

        user_string    db 50 dup(?)      ; Byte 2+: The string data


data ends
```

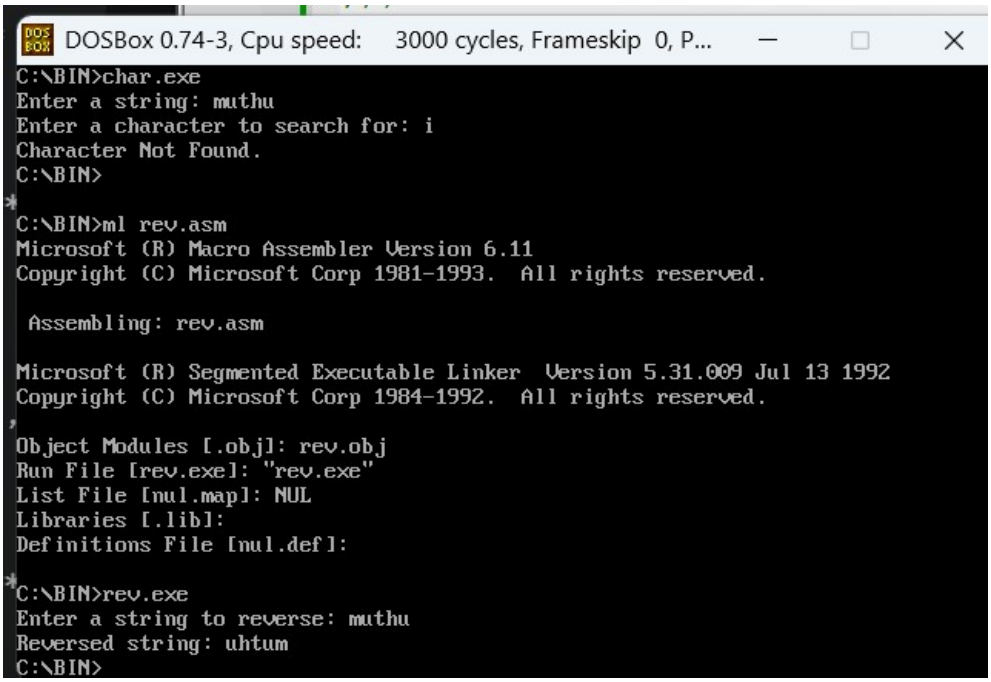
```
;--------------------------------------------------------------------------


;--------------------------------------------------------------------------
;               CODE SEGMENT
;--------------------------------------------------------------------------
code segment
assume cs:code, ds:data


start:


        ;-------------------------------------------------
        ; Initialize Data Segment
        ;-------------------------------------------------
        mov ax, data
        mov ds, ax


        ;-------------------------------------------------
        ; Prompt user for string
        ;-------------------------------------------------
        mov dx, offset prompt
        mov ah, 09h
        int 21h


        ;-------------------------------------------------
        ; Read string from user
        ;-------------------------------------------------
```

```
    mov dx, offset input_buffer
    mov ah, 0Ah
    int 21h

    ;------------------------------------------------
    ; Check if string is empty
    ;------------------------------------------------
    mov cl, actual_len          ; Load actual length of typed string into CL
    cmp cl, 0                   ; Did the user just press Enter?
    je exit_program             ; If yes, just exit

    mov ch, 0                   ; Clear CH so CX contains the full 16-bit length
    lea si, user_string         ; Point SI to the start of the typed string

    ;------------------------------------------------
    ; Step 1: PUSH all characters onto the stack
    ;------------------------------------------------
push_loop:
    mov al, [si]                ; Get a character from the string
    mov ah, 0                   ; Clear AH (because we must push a 16-bit AX register)
    push ax                     ; Push the character onto the stack
    inc si                      ; Move to the next character in the string
    loop push_loop              ; Repeat until CX becomes 0

    ;------------------------------------------------
    ; Step 2: POP characters back into the string
```

```
    ;-------------------------------------------------
    mov cl, actual_len          ; Reset CX back to the string length
    mov ch, 0
    lea si, user_string         ; Reset SI back to the start of the string

pop_loop:
    pop ax                      ; Pop the top character from the stack
    mov [si], al                ; Overwrite the string memory with the reversed character
    inc si                      ; Move to the next memory slot
    loop pop_loop               ; Repeat until CX becomes 0

    ; Add '$' at the end of the newly reversed string so DOS knows where to stop printing
    mov byte ptr [si], '$'

    ;-------------------------------------------------
    ; Print the result message
    ;-------------------------------------------------
    mov dx, offset msg
    mov ah, 09h
    int 21h

    ;-------------------------------------------------
    ; Print the reversed string
    ;-------------------------------------------------
    mov dx, offset user_string
    mov ah, 09h
```

```
        int 21h

        ;-------------------------------------------------

        ; Exit program

        ;-------------------------------------------------

exit_program:

        mov ah, 4Ch

        int 21h

code ends

end start

;-------------------------------------------------------------------------
```

OUTPUT:



TITLE: - PROGRAM TO COUNT 1S IN A BYTE

AIM: -

To write a ALP program to count 1’s in a byte.

ALGORITHM:-

- Initialize the data segment register.
- Display prompt to enter a character.
- Read a character using DOS function 01h.
- Store ASCII value in register AL.
- Set counter CX = 8 for 8 bits.
- Initialize bit count register BL = 0.
- Shift AL left by one bit.
- If Carry Flag is set, increment BL.
- Repeat steps 8–9 for all 8 bits using loop.
- Display result message.
- Convert count to ASCII and display it.

PROGRAM:

```
TITLE     BIT.ASM -- Program to count 1s in a byte


;//*****************************************************************************
;//* *
;//* Copyright (c) 1985-2022, American Megatrends International LLC.      *
;//* *
;//* All rights reserved. Subject to AMI licensing agreement.             *
;//* *
;//*****************************************************************************
;<AMI_FHDR_START>
;
```

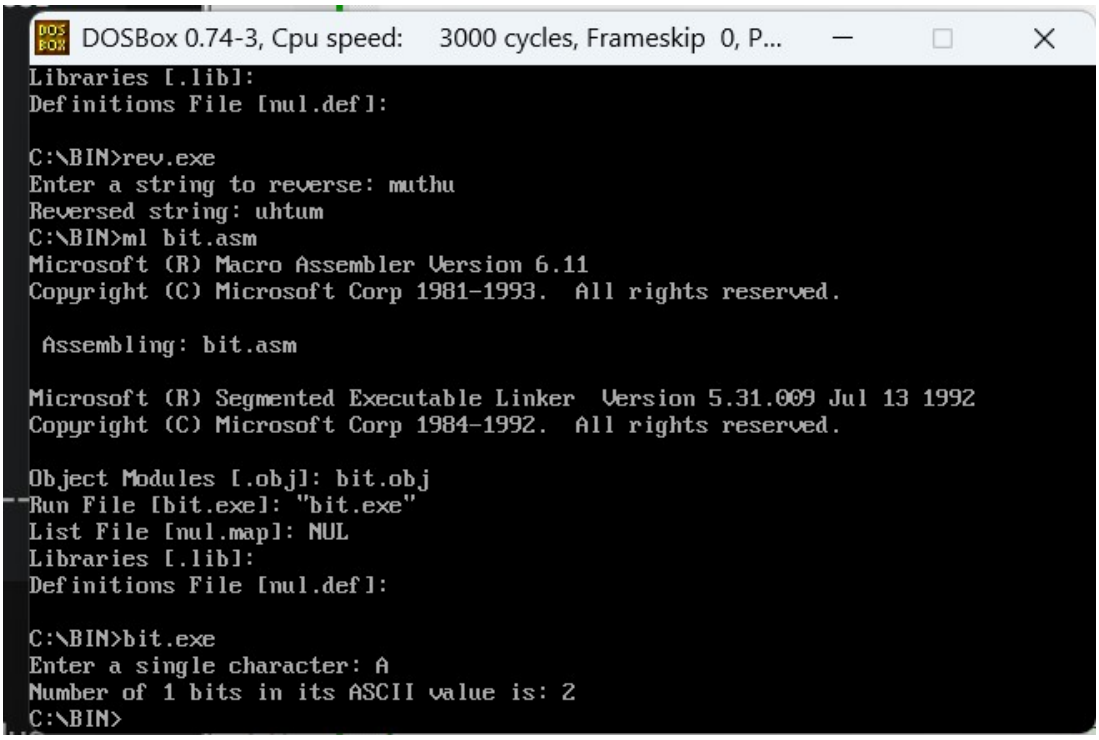
```
; Header:
; Author: Muthuganesh s
; Description: Program to count the number of 1 bits in a user-input byte
; Date: 02/04/2026
;
;<AMI_FHDR_END>
;*************************************************************************


.model small
.stack 100h


;-------------------------------------------------------------------------
;               DATA SEGMENT
;-------------------------------------------------------------------------
data segment


        prompt db "Enter a single character: $"
        msg        db 13, 10, "Number of 1 bits in its ASCII value is: $"


data ends
;-------------------------------------------------------------------------
;-------------------------------------------------------------------------
;               CODE SEGMENT
;-------------------------------------------------------------------------
code segment
assume cs:code, ds:data
```

```
start:

    ;-------------------------------------------------
    ; Initialize Data Segment
    ;-------------------------------------------------
    mov ax, data
    mov ds, ax


    ;-------------------------------------------------
    ; Prompt user for a character
    ;-------------------------------------------------
    mov dx, offset prompt
    mov ah, 09h
    int 21h


    ;-------------------------------------------------
    ; Read a single character from user
    ;-------------------------------------------------
    mov ah, 01h
    int 21h
    ; The ASCII value of the typed character is now in AL
    ;-------------------------------------------------
    ; Setup for counting bits
    ;-------------------------------------------------
    mov cx, 8                       ; Loop 8 times (since a byte has 8 bits)
    mov bl, 0                       ; BL will be our counter for the '1's
    ;-------------------------------------------------
```

```
    ; Loop to shift and count
    ;-------------------------------------------------
count_loop:
    shl al, 1                   ; SHL shifts all bits in AL left by 1.
                                    ; The leftmost bit falls into the Carry Flag (CF).
    jnc skip_increment          ; "Jump if No Carry": If CF is 0, skip the next line

    inc bl                       ; If CF is 1, increment our counter!
skip_increment:
    loop count_loop             ; Decrease CX. If CX > 0, jump back to count_loop
    ;-------------------------------------------------
    ; Display the result message
    ;-------------------------------------------------
    mov dx, offset msg
    mov ah, 09h
    int 21h
    ;-------------------------------------------------
    ; Display the count
    ;-------------------------------------------------
    ; Because a byte only has 8 bits, our max count is 8.
    ; This means we don't need complex division to print it!
    ; We just add '0' (30h) to convert it to an ASCII character.

    mov dl, bl                   ; Move our count from BL to DL for printing
    add dl, '0'                 ; Convert the number to an ASCII digit ('0' to '8')
```

```
    mov ah, 02h                    ; DOS interrupt to print a single character
    int 21h
    ;-------------------------------------------------
    ; Exit program
    ;-------------------------------------------------
    mov ah, 4Ch
    int 21h
code ends
end start
;-------------------------------------------------------------------------
```

OUTPUT:



TITLE: - COMAPARE TWO STRINGS

AIM: -

To write a ALP program to compare two strings

PROGRAM:

```
TITLE     COM.ASM -- Compare two strings entered by the user
;//**********************************************************************
;//* *
;//* Copyright (c) 1985-2022, American Megatrends International LLC.      *
;//* *
;//* All rights reserved. Subject to AMI licensing agreement.             *
;//* *
;//**********************************************************************
;<AMI_FHDR_START>
;
; Header:
; Author: Muthuganesh s
; Description: Take two strings from user and compare them
; Date: 02/04/2026
;
;<AMI_FHDR_END>
;**********************************************************************


.model small
.stack 100h


;---------------------------------------------------------------------------
;             DATA SEGMENT
;---------------------------------------------------------------------------
```

```
data segment

    prompt1 db "Enter the first string: $"
    prompt2 db 13, 10, "Enter the second string: $"

    msg_eq    db 13, 10, 13, 10, "Result: Strings are EQUAL!$"
    msg_neq db 13, 10, 13, 10, "Result: Strings are NOT EQUAL.$"

    ;-------------------------------------------------
    ; Buffer for String 1
    ;-------------------------------------------------
    buffer1 db 50                ; Byte 0: Max length allowed
    len1       db ?                 ; Byte 1: Actual length typed
    str1       db 50 dup(?)     ; Byte 2+: The string data

    ;-------------------------------------------------
    ; Buffer for String 2
    ;-------------------------------------------------
    buffer2 db 50                ; Byte 0: Max length allowed
    len2       db ?                 ; Byte 1: Actual length typed
    str2       db 50 dup(?)     ; Byte 2+: The string data

data ends
;--------------------------------------------------------------------------

;--------------------------------------------------------------------------
```

```
;               CODE SEGMENT

;-------------------------------------------------------------------------

code segment

assume cs:code, ds:data


start:


        ;-------------------------------------------------

        ; Initialize Data Segment

        ;-------------------------------------------------

        mov ax, data

        mov ds, ax


        ;-------------------------------------------------

        ; 1. Get First String

        ;-------------------------------------------------

        mov dx, offset prompt1

        mov ah, 09h

        int 21h


        mov dx, offset buffer1

        mov ah, 0Ah

        int 21h


        ;-------------------------------------------------

        ; 2. Get Second String
```

```
;-------------------------------------------------
mov dx, offset prompt2
mov ah, 09h
int 21h

mov dx, offset buffer2
mov ah, 0Ah
int 21h

;-------------------------------------------------
; 3. Compare Lengths First (Optimization)
;-------------------------------------------------
mov al, len1            ; Load length of first string
cmp al, len2            ; Compare with length of second string
jne not_equal           ; If lengths are different, strings are definitely not equal

;-------------------------------------------------
; 4. Prepare for Character Comparison
;-------------------------------------------------
mov cl, len1            ; Use the length as our loop counter
cmp cl, 0               ; Are both strings completely empty? (User just pressed Enter twice)
je equal                ; If both are empty, technically they are equal!

mov ch, 0               ; Clear CH so CX has the correct 16-bit loop count
lea si, str1            ; Point SI to first string data
lea di, str2            ; Point DI to second string data
```

```
    ;-------------------------------------------------
    ; 5. Compare Character by Character
    ;-------------------------------------------------
compare_loop:
    mov al, [si]            ; Get char from string 1
    mov bl, [di]            ; Get char from string 2
    cmp al, bl              ; Compare them
    jne not_equal           ; If a mismatch is found, exit immediately to 'not_equal'
    inc si                  ; Move to next char in string 1
    inc di                  ; Move to next char in string 2
    loop compare_loop       ; Decrease CX, and repeat if CX > 0
    ;-------------------------------------------------
    ; If the loop finishes naturally, all chars matched!
    ;-------------------------------------------------
equal:
    mov dx, offset msg_eq
    mov ah, 09h
    int 21h
    jmp exit_program


    ;-------------------------------------------------
    ; If lengths differ or a char mismatch was found
    ;-------------------------------------------------
not_equal:
    mov dx, offset msg_neq
```

```
    mov ah, 09h
    int 21h
    ;-------------------------------------------------
    ; Exit program
    ;-------------------------------------------------
exit_program:
    mov ah, 4Ch
    int 21h
code ends
end start
;-------------------------------------------------------------------------------
```

OUTPUT: